CHAPTER 3

Functions

The need for repeatable sets of instructions was recognized early in computing history. The 1950s onwards saw languages developed (such as FORTRAN or ALGOL) that supported *procedures*. Procedures (also known as *routines* or *sub-routines*) are reusable series of computational steps that have a mechanism for passing control to and from them. The formal mathematical basis for this kind of computation – *Lambda Calculus* – was developed in the 1930s by Alonzo Church. Church's work was based on the mathematical concept of *functions*, which are a way of conceptualizing dependencies between variables. In mathematics, functions take values (as input), perform operations, and return new values (as output).

Some languages (such as Visual Basic) distinguish between sub-routines (that don't return values) and functions (that do return values). PHP – like many others – simply uses the `function` keyword regardless of whether the routine accepts or returns values. In computing the words *routine*, *sub-routine*, and *function* are often generalized as *callable unit*. (Remember PHP's `callable` data type from Chapter 2, "Data, Types, and Variables.")

As programs grow in size and complexity, it is inevitable that repetition will creep in. Tasks often need to be performed multiple times, or the same complex logic needs to be applied in multiple places. There is a principle of software development called *DRY* (an acronym for "Don't Repeat Yourself") which, in *The Pragmatic Programmer: From Journeyman to Master* (Hunt & Thomas, 1999), stipulates that "Every piece of knowledge must have a single, unambiguous, authoritative representation within a system." This broad statement can apply to much more than just the code you write (e.g., documentation, build systems, or database schemas). It should naturally occur to most developers that it is sub-optimal to reproduce the same code in multiple places.

Imagine an application that, during its lifecycle, sends multiple emails. You could just write out (or copy/paste) all the code needed to interact with an SMTP server at every point that mail needs to be sent. Let's say that each time this happens you need fifteen lines of code. Now multiply that by the number of times mail is sent and

© Paul Tregoing 2025
P. Tregoing, *Exploring PHP 8*, https://doi.org/10.1007/979-8-8688-2136-3_3